

T3 Stack Overview

What the T3 Stack Is

The T3 Stack is a full-stack TypeScript architecture designed around **end-to-end type safety**. Frontend, backend, and database layers all share types, reducing runtime bugs and duplicated logic.

Core technologies used:

- **Next.js** — routing, rendering, server/client components
- **tRPC** — type-safe API calls without REST or GraphQL
- **Prisma** — database ORM with generated TypeScript types
- **BetterAuth** — authentication and session management
- **TypeScript** — shared type system across the stack

High-Level Architecture

- UI components call **tRPC** procedures directly
- **tRPC** procedures run on the server and contain business logic
- Prisma is used inside **tRPC** procedures to access the database
- **BetterAuth** integrates with Prisma for users and sessions

General Folder Structure

Typical T3 project using the Next.js App Router:

```
.
├── prisma/
│   └── schema.prisma          % database models
├── src/
│   └── app/
│       ├── layout.tsx         % root layout + providers
│       ├── page.tsx           % home page
│       └── api/
│           ├── trpc/
│           │   └── route.ts    % tRPC HTTP handler
│           ├── auth/
│           │   └── route.ts    % BetterAuth route
│           └── server/
│               ├── api/
│               │   ├── root.ts    % main tRPC router
│               │   └── routers/   % feature routers
│               ├── auth.ts        % BetterAuth config
│               ├── db.ts          % Prisma client
│               └── trpc/
│                   ├── react.tsx  % tRPC React hooks
│                   └── env.mjs     % typed env variables
├── .env
└── next.config.mjs
```

End-to-End Data Flow Example

This section shows how data flows through the T3 Stack: database → server API → client UI, with typical file locations.

1. Prisma (Database Layer)

File: `prisma/schema.prisma`

Prisma defines the database schema and generates a typed client.

```
model Post {
  id          String    @id @default(cuid())
  title       String
  createdAt   DateTime  @default(now())
}
```

Important fields:

- `Post` — database table represented as a TypeScript type
- `@id` — primary key
- `@default` — values generated automatically

File: `src/server/db.ts`

Using Prisma in server code:

```
export const prisma = new PrismaClient();
```

Key variable:

- `prisma` — ORM client used for all database access

2. tRPC (Server API Layer)

File: `src/server/api/routers/post.ts`

tRPC exposes server-side logic as typed procedures.

```
export const postRouter = createTRPCRouter({
  getAll: publicProcedure.query(({ ctx }) => {
    return ctx.db.post.findMany();
  }),
});
```

Important variables:

- `postRouter` — groups related procedures
- `publicProcedure` — procedure callable without auth
- `ctx` — request context
- `ctx.db` — Prisma client provided to tRPC

File: `src/server/api/root.ts`

Routers are combined into a single API.

```
export const appRouter = createTRPCRouter({
  post: postRouter,
});
```

3. Client (Frontend Layer)

File: `src/trpc/react.tsx`

The typed tRPC client is generated from the server router.

```
export const api = createTRPCReact<AppRouter>();
```

File: `src/app/page.tsx`

Calling a tRPC procedure from a React component:

```
const { data, isLoading } = api.post.getAll.useQuery();
```

Important variables:

- `api` — typed client derived from server routers
- `post.getAll` — mirrors server procedure name
- `data` — fully typed database result
- `isLoading` — request lifecycle state

Key Takeaway

Each layer builds on the previous one:

`schema.prisma` → `tRPC routers` → `React hooks`

Types are written once and enforced everywhere.